

PBE DES Encryption and Decryption Time Analysis

COMP232 | March 14, 2026

Zhibin Jiang 201945844

1 Task A PBE DES Encryption and Decryption Time

In this experiment, we first establish baseline time consumption data for encryption and decryption. The experiment utilizes the PBESWithMD5AndDES algorithm, with the specific parameter configuration as follows:

- **Iteration Count:** 1024
- **Loop Count:** 10000 (used to obtain stable average time consumption)
- **Preset Plaintext:** This is a secret message for COMP232!
- **Initial Salt (Salt A):**

```
{(byte) 0xc7, (byte) 0x73, (byte) 0x21, (byte) 0x8c, (byte) 0x7e, (byte) 0xc8, (byte) 0xee, (byte) 0x99}
```

1.1 Experimental Results

Under the initial parameters, the average encryption and decryption time consumption for the four target passwords is shown in the table below:

Table 1: Encryption and Decryption Time Results (Original Salt)

Password Plaintext	Encryption Time (ms)	Decryption Time (ms)	Total Time (ms)
P@S\$W0rD	0.126273	0.127715	0.253988
thisismypassword	0.127971	0.125547	0.253517
VeryLongP@\$\$W0rD	0.126662	0.126698	0.253360
%0^t#2Fv0JUjVdRV2RW%	0.129826	0.125737	0.255562

Additionally, we can experiment with the impact of different Salts on computational performance; we modified the Salt to: **Modified Salt:**

```
{(byte) 0xc6, (byte) 0x72, (byte) 0x20, (byte) 0x8b, (byte) 0x7d, (byte) 0xc7, (byte) 0xed, (byte) 0x98}
```

Table 2: Encryption and Decryption Time Results (Modified Salt)

Password Plaintext	Encryption Time (ms)	Decryption Time (ms)	Total Time (ms)
P@S\$W0rD	0.127133	0.127085	0.254217
thisismypassword	0.126753	0.125663	0.252416
VeryLongP@\$\$W0rD	0.128006	0.126938	0.254944
%0^t#2Fv0JUjVdRV2RW%	0.130005	0.125659	0.255664

By comparing the two sets of data, the following conclusions can be drawn:

1. **Salt Independence:** The experiment proves that changing the salt value has almost no impact on the computational overhead of encryption and decryption. The minor fluctuations in the two sets of data are attributed to system measurement errors.

2. **Subsequent Prediction:** Based on this observation, we predict that the performance of the encryption system is primarily constrained by the iteration count. In the subsequent Task C, we will analyze the linear growth relationship between time consumption and the iteration count in detail.

We will use the first salt value for the following tasks.

2 Task B: Brute Force Attack Time Estimation

In this section, we assume that the attacker knows all encryption parameters except for the password:

- **Predefined Plaintext**
- **Resulting Ciphertext**
- **Salt**
- **Iteration Count**

Based on the experimental data from Task A, the average time consumption for a single decryption process is $t = 0.12$ ms. The attacker's goal is to match the known plaintext and ciphertext by exhausting the password space.

The total time required for brute force cracking depends on the password length (L) and the character set space (R). The formula for estimating the average expected time to successfully recover a password is as follows:

$$T = \frac{R^L \times t}{2} \quad (1)$$

- T : Estimated average cracking time.
- R : Password space (character set combinations).
- L : Password length.
- t : Time consumption for a single decryption (taking the experimental average of 0.12 ms).
- **Factor 1/2:** Based on statistical principles, an attacker can typically successfully discover the password after traversing 50% of the search space.

Definition of Password Space (R): To cover different attack scenarios, we define the possible password character combinations as follows:

In actual attacks, the cracker usually cannot predict the exact composition of the password, so they will adopt a progressive trial strategy. For passwords containing complex characters (such as P@S\$W0rD), an attacker might first try subset spaces of pure letters or pure digits, and then expand to the full character set ($R = 95$) after failure. Since subset spaces (such as 26^8) are almost negligible compared to the order of magnitude of the full set space (such as 95^8), our estimation will directly use the **minimum coverage space** to which the password belongs as the calculation baseline.

According to the password list provided in Experiment 1, under the condition of $t = 0.12$ ms, the estimated cracking time is shown in the table below:

Conclusion: The results indicate that **password length (L)** is the core factor determining the strength against brute force cracking. Even though the character complexity of `thisismypassword` is low, its security far exceeds that of an 8-character complex password because its length reaches 16 digits.

Table 3: Password Space Definitions

Combinations	Password Space R
Lowercase	26
Uppercase	26
Digits	10
Special Characters	33
Lowercase + Uppercase	52
Lowercase + Digits	36
Lowercase + Special Characters	59
Uppercase + Digits	36
Uppercase + Special Characters	59
Digits + Special Characters	43
Lowercase + Uppercase + Digits	62
Lowercase + Uppercase + Special Characters	85
Lowercase + Digits + Special Characters	69
Uppercase + Digits + Special Characters	69
Lowercase + Uppercase + Digits + Special Characters	95

Table 4: Estimated Brute Force Cracking Time

Password Plaintext	Length (L)	Combination Space (R)	Estimated Time (Years)
P@S\$W0rD	8	Full Character Set (95)	≈ 12622
thisismypassword	16	Pure Lowercase (26)	$\approx 8.2 \times 10^{10}$
VeryLongP@\$\$W0rD	16	Full Character Set (95)	$\approx 8.37 \times 10^{19}$
%0^t#2Fv0JUjVdRV2RW%	20	Full Character Set (95)	$\approx 6.82 \times 10^{27}$

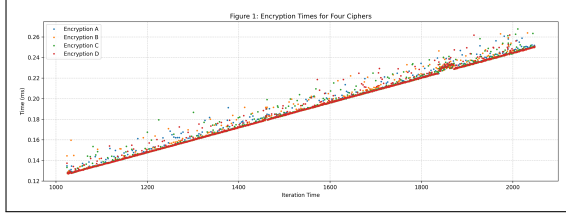
3 Task C: Analysis of the Impact of Iteration Count on Encryption and Decryption Time

Explore the patterns of how attack costs change with Iteration Count in PBE:

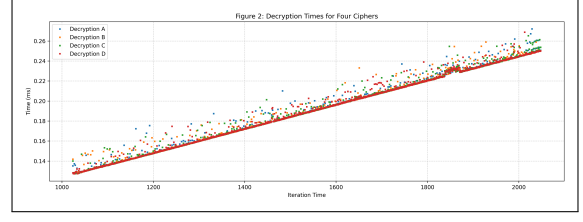
- **Test Range:** The iteration count gradually increases from 1024 to 2048.
- **Sampling Frequency:** For each iteration point, the program executes 1000 encryption and decryption cycles and takes the average. 1000 cycles are sufficient to eliminate random system fluctuations in a statistical sense and clearly demonstrate the trend.

Through data collection for the four test passwords (Decryption A, B, C, D), we obtained the distribution plots as shown in Figure 1. Specifically, Figure 1a and Figure 1b illustrate the raw performance times, while Figure 1c and Figure 1d provide the corresponding curve fitting analysis.

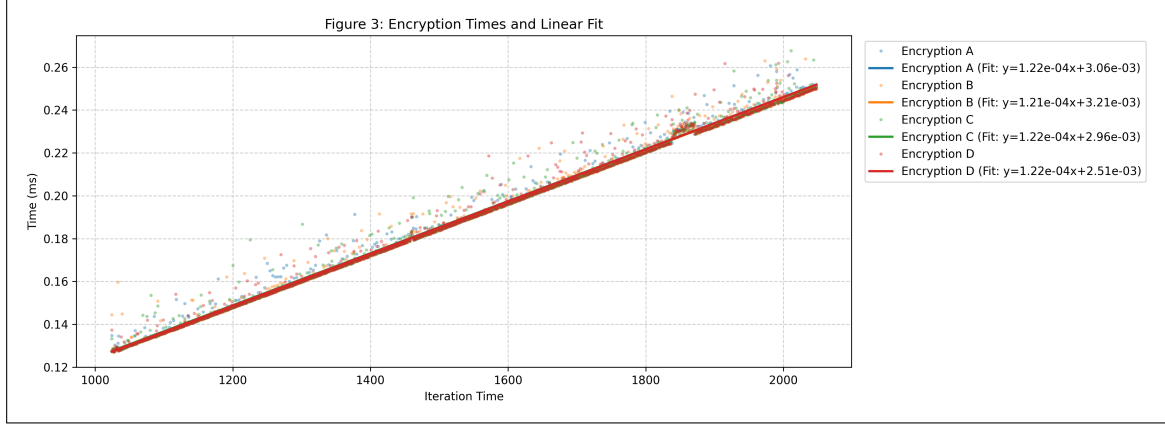
- **Original Data Distribution** (see Figures 1 and 2): As the iteration count increases, the encryption and decryption time consumption for all passwords shows an extremely consistent upward trend.
- **Linear Fitting Analysis** (see Figures 3 and 4): Using the least squares method to perform a linear function fit on the sampling points, we obtain the following analytical expression (taking Decryption A as an example):



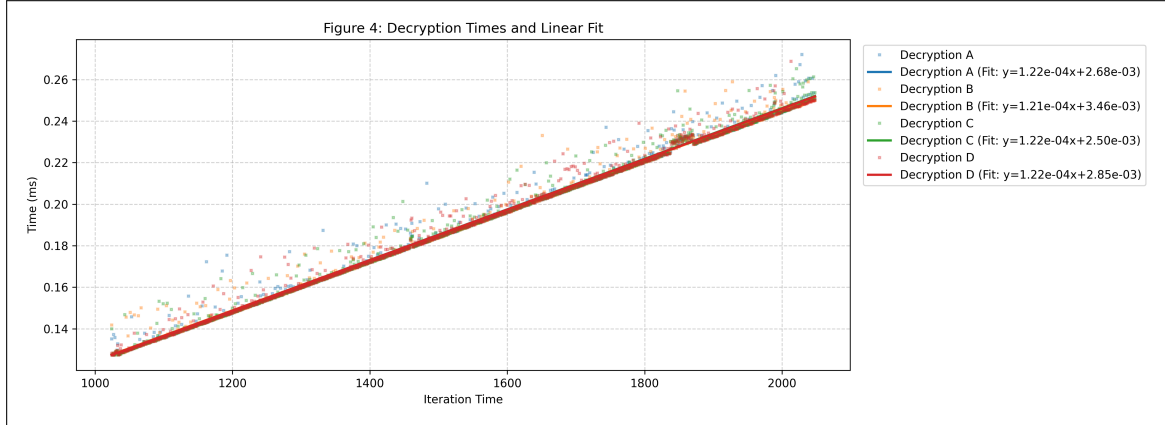
(a) Encryption Performance Times



(b) Decryption Performance Times



(c) Encryption Curve Fitting



(d) Decryption Curve Fitting

Figure 1: Distribution Plots and Fitting Analysis for Decryption Tasks A-D

$$y = (1.22 \times 10^{-4})x + (2.68 \times 10^{-3}) \quad (2)$$

Based on the fitted images and functional analytical expressions, we can draw the following key conclusions:

1. **High Linear Correlation:** The slope of the fitted line $a \approx 1.22 \times 10^{-4}$ has a high degree of consistency, proving that a rigorous linear positive correlation exists between time consumption y and iteration count x .
2. **Unit Iteration Cost:** The slope a represents the additional computational time overhead (approximately $0.122 \mu s$) brought by each added hash iteration. This overhead is generated by the repetitive execution of hash operations by the underlying algorithm (such as MD5)!

3. **Meaning of the Intercept Term:** The intercept b (approximately 0.00268 ms) represents the algorithm's fixed initialization overhead, including memory allocation, `Cipher` object instantiation, etc. Since $b \ll ax$, it can be confirmed that iteration operations are the absolute dominant factor in PBE performance loss.

This linear growth relationship is the theoretical basis for **Key Stretching** technology. By increasing the iteration count, the system can mandatorily double the attacker's total cracking time at a minimal cost (such as an increase of 0.5 ms) that is almost imperceptible to legitimate users. This produces a strong defensive gain when facing the massive search space discussed in Task B.

4 Task D: Attack Estimation Under Unknown Iteration Count

In Task B, we assume that the attacker knows the iteration count $n = 1024$; therefore, the single decryption time t is a fixed small constant (approximately 0.12 ms). However, in this variant, although the attacker possesses the salt, plaintext, and ciphertext, they **do not know the iteration count** N .

This means that during the brute-force process, for every possible password attempted, the attacker must **additionally traverse** all possible iteration counts until the generated ciphertext matches.

The fitting results of Task C have proven that the decryption time t has a linear relationship with the iteration count n :

$$t(n) = a \cdot n + b \quad (3)$$

Where a (slope) and b (intercept) are constants measured experimentally.

When the iteration count is unknown, the total time t to attempt a single password is no longer $t(1024)$, but the **cumulative sum** within the possible range $[1, N_{max}]$:

$$t(N_{max}) = \sum_{n=1}^{N_{max}} (a \cdot n + b) \quad (4)$$

Using the summation formula for arithmetic progressions, we can expand the above formula:

$$t(N_{max}) = a \cdot \frac{N_{max}(N_{max} + 1)}{2} + b \cdot N_{max} \quad (5)$$

This means that when the iteration count is unknown, the time complexity for attacking a single password changes from $O(1)$ to a growth level of $O(N_{max}^2)$ (relative to the upper limit of the iteration range).

4.1 Estimation Analysis (Example)

Originally, decrypting P@\$\$W0rD once only required $t = 0.12$ ms. Now, **referring to the fitting function results from Task C**, we know the analytical expression for decryption time:

$$t = 0.000122n + 0.00268 \quad (6)$$

Substituting $n = 1024$ here yields $t = 0.1276$ ms.

Here $a = 0.000122$ and $b = 0.00268$. Now, if the iteration count is unknown, we need to assume a reasonable upper limit for the iteration count $N_{max} = 10,000$.

Then the single decryption time becomes:

$$t(N_{max}) = a \cdot \frac{N_{max}(N_{max} + 1)}{2} + b \cdot N_{max}$$

$$t(10000) = 0.000122 \times \frac{10000 \times 10001}{2} + 0.00268 \times 10000 = 6127.41$$

The time changed from $t = 0.1276$ ms to $t = 6127.41$ ms, an increase of nearly 48,000 times!

The corresponding total cracking time will also increase by 48,000 times!

Conclusion: Unknown iteration counts greatly increase the computational cost of brute-force attacks. On top of the password space R^L , the attacker is forced to introduce a second search dimension. This proves that **hiding algorithm parameters** (such as iteration count) is just as important as increasing the values of the parameters themselves!

5 Task E: Comparative Analysis and Explanation of Differences

In this section, we compare the local estimated time measured in Task B with industry-recognized online password strength assessment services (such as Security.org) to evaluate the gap between theoretical models and real-world attack scenarios.

The following table displays the significant differences between the local single-threaded CPU execution environment ($t = 0.12$ ms) and professional security assessment tools:

Table 5: Comparison Between Local and Online Tool Estimates

Target Password	Local Est. Time (Years)	Security.org Est. Time	Magnitude Diff
P@S\$W0rD	$\approx 1.26 \times 10^4$	8 Hours	$\approx 10^7$
thisismypassword	$\approx 8.30 \times 10^{10}$	$\approx 3 \times 10^3$ Years	$\approx 10^7$
VeryLongP@\$\$W0rD	$\approx 8.37 \times 10^{19}$	$\approx 1 \times 10^{12}$ Years	$\approx 10^7$
%0^t#2Fv0JUjVdRV2RW%	$\approx 6.82 \times 10^{27}$	$\approx 4.2 \times 10^{19}$ Years	$\approx 10^8$

Experimental results show that the local estimated time is generally 10^7 to 10^8 orders of magnitude slower than online tools. This massive difference primarily stems from the following factors:

- Local experiments only use a **single thread** of an Intel i9-12900H for linear computation. In contrast, professional attackers utilize clusters composed of hundreds of GPUs (graphics cards), or even specially designed ASIC chips for parallel attacks.
- **Throughput Difference:** The throughput of a GPU when processing hash and encryption operations is tens of thousands of times higher than that of a CPU. This parallel processing capability can potentially compress a workload that originally required “ten thousand years” into the “hour” level.

Furthermore, consider data breaches.

- **Known Leaks:** As proven in Lab 1, common passwords like P@S\$W0rD and thisismypassword have long appeared on the Kaspersky website, indicating they have been leaked in databases.
- **Non-Brute-Force Exhaustion:** Online tools (and hackers) usually prioritize **dictionary attacks**. If a password is in a leaked database, the cracking time is almost zero, without having to go through the full brute-force exhaustion process simulated in Task 2.

In addition, there are hardware differences:

- For the hash iteration process in the PBESWithMD5AndDES algorithm, specific hardware (such as FPGA) can perform deep pipeline optimization. This allows the iteration cost $(a \cdot n + b)$ observed in Task C to be significantly slashed on specialized hardware, further lowering the upper bound of the cracking time.

5.1 Conclusion

Although the cracking time derived from local single-thread simulations shows extremely high security mathematically, in real-world threat models, the multi-core parallel computing power of attackers and existing leaked dictionaries must be considered. Experiments prove that **increasing iteration counts** and **using long random passwords** remain the most effective lines of defense against high-performance hardware cracking.

A Data Support and Replication

Server: @lxfarm05.csc.liv.ac.uk

A.1 Java Version

```
openjdk 21.0.10 2026-01-20 LTS
OpenJDK Runtime Environment (Red_Hat-21.0.10.0.7-1) (build 21.0.10+7-LTS)
OpenJDK 64-Bit Server VM (Red_Hat-21.0.10.0.7-1) (build 21.0.10+7-LTS, mixed mode, sharing)
```

A.2 Python Version (Images)

```
Python 3.13.10
```

A.3 CPU Information

```
CPU Model: 12th Gen Intel(R) Core(TM) i9-12900H
Architecture: x86_64
CPU Cores/Threads: 14 Cores / 20 Threads
Max Frequency: 5000.00 MHz
L3 Cache size: 24 MiB
Instruction Set Extensions: AES, SHA_NI, AVX2, AVX_VNNI
Virtualization: VT-x
```

B Import Libraries

Running the following code requires importing the following libraries:

```
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.PBEParameterSpec;
import java.io.BufferedWriter;
import java.io.FileWriter;
import java.math.BigDecimal;
import java.math.BigInteger;
import java.math.RoundingMode;
```

C Constants

All the tasks use the same Salt values, passwords and plaintext.

```
public static final byte[] salt = {
    (byte) 0xc7, (byte) 0x73,
```

```

        (byte) 0x21, (byte) 0x8c,
        (byte) 0x7e, (byte) 0xc8, (byte) 0xee,
        (byte) 0x99};
public static final String[] passwordList = {
    "P@S$W0rD",
    "thisismypassword",
    "VeryLongP@$$W0rD",
    "%0~t#2Fv0JUjVdRV2RW%"};
public static final String plainText =
    "This is a secret message for COMP232.";

```

D Encryption and Decryption Methods

Some functions mentioned in this article: encryption functions, decryption functions.

All use DES and are based on Password-Based Encryption (PBE), using MD5.

```

public static byte[] encode(String password, int iterationTime, byte[] salt, String text) throws
    Exception {
    SecretKey key = SecretKeyFactory.
        getInstance("PBEWithMD5AndDES").
        generateSecret(new PBEKeySpec(password.toCharArray(), salt, iterationTime));
    Cipher pbeCipher = Cipher.getInstance("PBEWithMD5AndDES");
    pbeCipher.init(Cipher.ENCRYPT_MODE, key, new PBEPARAMETER_SPEC(salt, iterationTime));
    return pbeCipher.doFinal(text.getBytes());
}

public static byte[] decode(String password, int iterationTime, byte[] salt, byte[] cipherText) throws
    Exception {
    SecretKey key = SecretKeyFactory.
        getInstance("PBEWithMD5AndDES").
        generateSecret(new PBEKeySpec(password.toCharArray(), salt, iterationTime));
    Cipher pbeCipher = Cipher.getInstance("PBEWithMD5AndDES");
    pbeCipher.init(Cipher.DECRYPT_MODE, key, new PBEPARAMETER_SPEC(salt, iterationTime));
    return pbeCipher.doFinal(cipherText);
}

```

E Task A Code

Calling this method will output the encryption and decryption times for the four passwords in the Terminal. Its basic working principle is to run `loopCount = 10000` rounds for each password and take the average to calculate stable time consumption.

```

public static void taskA() throws Exception{
    int iterations = 1024; // Iteration Time
    int loopCount = 10000; // loop to ensure stability
    // Warm up - without timing to make sure JVM loaded the .class and JIT.
    for (int i = 0; i < 5000; i++)
        decode(passwordList[0], iterations, salt, encode(passwordList[0], iterations, salt, plainText));
    System.out.printf("%-25s | %-12s | %-12s | %-12s\n", "Password", "Encrypt(ms)", "Decrypt(ms)", "
        Total(ms)");
    System.out.println("-----");
    for (String password : passwordList) {
        // measure encode time
        long startEncryptTime = System.nanoTime();
        byte[] cipherText = null;
        for (int i = 0; i < loopCount; i++)
            cipherText = encode(password, iterations, salt, plainText);
        double avgEncryptTime = (System.nanoTime() - startEncryptTime) / (double) loopCount / 1_000_000
            .0;
        // measure decode time
        long startDecrypt = System.nanoTime();

```



```

        for (int i = 0; i < loopCount; i++) decode(password, iterations, salt, cipherText);
        double avgDecryptTime = (System.nanoTime() - startDecrypt) / (double) loopCount / 1_000_000.0;
        // total time
        System.out.printf("%-25s | %-12.6f | %-12.6f | %-12.6f%n",
                           password, avgEncryptTime, avgDecryptTime, avgEncryptTime + avgDecryptTime);
    }
}

```

E.1 Output

Password	Encrypt(ms)	Decrypt(ms)	Total(ms)
P@S\$W0rD	0.125848	0.124782	0.250630
thisismypassword	0.127891	0.124878	0.252769
VeryLongP@\$\$W0rD	0.125202	0.124232	0.249434
%0~t#2Fv0JUjVdRV2RW%	0.126802	0.126409	0.253211

F Task B Code

Task B is mainly used to analyze and model the time for password brute-force cracking.

```

public static void taskB() throws Exception{
    calculateBruteForce("P@S$W0rD | lc+uc+num+char",95,8,0.12);
    calculateBruteForce("thisismypassword | lc ",26,16,0.12);
    calculateBruteForce("VeryLongP@$$W0rD | lc+uc+num+char",95,16,0.12);
    calculateBruteForce("%0~t#2Fv0JUjVdRV2RW% | lc+uc+num+char",95,20,0.12);
}

public static void calculateBruteForce(String label, int space, int length, double msPerAttempt) {
    BigInteger attempts = BigInteger.valueOf(space).pow(length);
    // search half space is possible to find out the password
    BigDecimal totalMs = new BigDecimal(attempts)
        .multiply(BigDecimal.valueOf(msPerAttempt))
        .divide(BigDecimal.valueOf(2), RoundingMode.HALF_UP);
    BigDecimal days = totalMs.divide(BigDecimal.valueOf(86_400_000), 2, RoundingMode.HALF_UP);
    BigDecimal years = days.divide(BigDecimal.valueOf(365), 2, RoundingMode.HALF_UP);
    System.out.printf("Password info: %-30s | Length: %2d | Estimate Year : %s%n", label, length, years
        .toPlainString());
}

```

F.1 Output

```

Password info: P@S$W0rD | lc+uc+num+char | Length: 8 | Estimate Year : 12622.15
Password info: thisismypassword | lc | Length: 16 | Estimate Year : 82969449960.86
Password info: VeryLongP@$$W0rD | lc+uc+num+char | Length: 16 | Estimate Year : 83737950656728633515.11
Password info: %0~t#2Fv0JUjVdRV2RW% | lc+uc+num+char | Length: 20 | Estimate Year :
6820508417209707655201872915.62

```

G Task C Code

Used for plotting charts, analyzing linear relationships, and generating a data.txt file in the current directory.

```

public static void taskC() throws Exception {
    BufferedWriter br = new BufferedWriter(new FileWriter("data.txt"));
    int startIter = 1024;
    int endIter = 2048;
    int loopCount = 1000; // 1000 is enough
    // warm up runtime environment
    for (int i = 0; i < 2000; i++)
        decode(passwordList[0], 1024, salt, encode(passwordList[0], 1024, salt, plainText));
}

```

```

System.out.println("Iteration, Pwd1_Dec(ms), Pwd2_Dec(ms), Pwd3_Dec(ms), Pwd4_Dec(ms)");
for (int iter = startIter; iter <= endIter; iter++) {
    br.write(iter + ",");
    for (int i = 0 ; i < passwordList.length ; i ++ ) {
        String password = passwordList[i];
        // encrypt time
        long startEncryptTime = System.nanoTime();
        byte[] cipherText = null;
        for (int k = 0; k < loopCount; k++)
            cipherText = encode(password, iter, salt, plainText);
        double avgEncryptTime = (System.nanoTime() - startEncryptTime) / (double) loopCount / 1
            _000_000.0;

        // decrypt time
        long startDecrypt = System.nanoTime();
        for (int k = 0; k < loopCount; k++) decode(password, iter, salt, cipherText);
        double avgDecryptTime = (System.nanoTime() - startDecrypt) / (double) loopCount / 1_000_000
            .0;

        // total time
        br.write(String.format("%.5f,%.5f",avgEncryptTime,avgDecryptTime));
        if(i!=passwordList.length-1)
            br.write(",");
    }
    System.out.println("Iter = " +iter);
    br.newLine();
    br.flush();
}
br.close();
}

```

G.1 Output

```

1024,(0.13483,0.13515),(0.14449,0.14183),(0.13328,0.14003),(0.13737,0.12818)
1025,(0.12746,0.12747),(0.12732,0.12734),(0.12845,0.12746),(0.12748,0.12740)
1026,(0.12754,0.12761),(0.12734,0.12819),(0.12869,0.12825),(0.12848,0.12819)
1027,(0.12830,0.13721),(0.12700,0.12746),(0.12717,0.12731),(0.12707,0.12724)
...

```

First element is `iteration time` where the remaining are respectively the pair for those 4 passwords' encryption time and decryption time.